MODULE 2 · TEXTBOOK 1 · CHAPTERS 3 & 4 · 9 HOURS

Graphics Output Primitives & Their Attributes

Coordinate reference frames, drawing points, lines and curves in OpenGL, and how attributes such as colour, thickness and style control the appearance of every primitive.

Coordinate Frames | Point, Line, Curve Functions | Colour & Gray Scale | Primitive Attributes

Module Roadmap

First we draw shapes, then we control how they look

Part A · Output Primitives (Ch 3)

- Coordinate reference frames
- 2D world coordinate frames in OpenGL
- OpenGL point functions
- OpenGL line functions
- OpenGL curve functions

Part B · Attributes (Ch 4)

- Colour and gray scale
- Point attributes
- Line attributes
- Curve attributes
- OpenGL point attribute functions
- OpenGL line attribute functions

Graphics Output Primitives

The basic geometric building blocks of any picture

Output primitives are the fundamental shapes and characters used to describe a scene. Complex pictures are built by combining many of them.

Point

A single position, the simplest primitive.

Line

A straight segment between two endpoints.

Curve

Circles, ellipses and splines.

Polygon

Filled areas and text.

GEOMETRY VS ATTRIBUTES

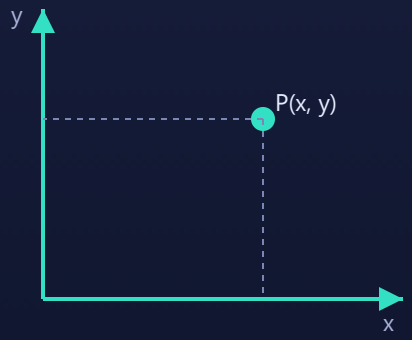
Geometry is where a primitive sits (its coordinates). **Attributes** are how it looks (colour, size, style). Module 2 covers both.

Coordinate Reference Frames

Every primitive is specified relative to a coordinate system

- **Cartesian coordinates**: the standard (x, y) 2D system.
- **Screen coordinates**: integer pixel positions, origin often at a corner.
- **World coordinates**: the frame the whole scene is defined in.
- A point is finally stored in the frame buffer at its device pixel location.

```
setPixel(x, y);          // turn on pixel (x,y)
getPixel(x, y, color);   // read a pixel colour
```



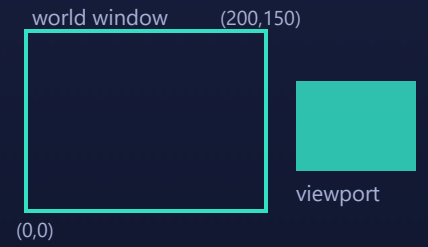
A point is an (x, y) pair in the chosen frame.

Specifying 2D World Coordinates in OpenGL

Define which region of the world maps onto the window

```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(xmin, xmax, ymin, ymax); // 2D clipping window
```

- `gluOrtho2D` sets the 2D **world coordinate window**, the rectangle of world space shown in the display window.
- Anything inside this range is visible; anything outside is clipped away.
- `glMatrixMode(GL_PROJECTION)` selects the projection matrix before you define the view.
- `glLoadIdentity()` resets it to a known state first.



gluOrtho2D(0,200,0,150) maps world space to the window.

OpenGL Point Functions

Plot one or more points inside glBegin(GL_POINTS)

```
glBegin(GL_POINTS);
    glVertex2i(50, 100);
    glVertex2i(75, 150);
    glVertex2i(100, 200);
glEnd();
```

- `glVertex*` specifies a coordinate position.
- Suffix: number of coordinates (2, 3, 4) plus data type (i, f, d, s).
- `glVertex2i` is 2 integer coordinates; `glVertex3f` is 3 floats.
- You can also pass an array: `glVertex2iv(point)` where v means vector.

Each glVertex call places one point.

OpenGL Line Functions

Three primitive modes decide how vertices connect

Separate segments, an open polyline, or a closed loop.

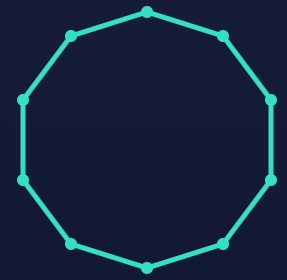
Primitive	Behaviour
`GL_LINES`	Each pair of vertices is a separate segment
`GL_LINE_STRIP`	Connected polyline: v0 to v1 to v2 and so on
`GL_LINE_LOOP`	Like strip, but also joins the last vertex back to the first

OpenGL Curve Functions

OpenGL has no direct circle primitive, so curves are approximated

- Circles, ellipses and arcs are drawn as a **series of short line segments**, a polyline.
- The **GLU library** provides functions for smooth curves and surfaces such as spheres, cylinders and disks.
- More points give a smoother curve but need more computation.

```
for (i = 0; i < n; i++) {
  theta = 2*PI*i/n;
  x = xc + r*cos(theta);
  y = yc + r*sin(theta);
  glVertex2f(x, y);
}
```



A circle is many short segments joined together.

GLU QUADRICS

`gluNewQuadric()`, `gluSphere()`, `gluCylinder()` and `gluDisk()` build curved surfaces for you.

Polygon Fill Area Primitives

Filled shapes are as common as line drawings

OpenGL offers several ways to fill areas.

Primitive	Meaning
`GL_POLYGON`	Single filled convex polygon
`GL_TRIANGLES`	Every 3 vertices form one triangle
`GL_TRIANGLE_STRIP`	Connected strip of triangles
`GL_TRIANGLE_FAN`	Triangles sharing the first vertex
`GL_QUADS`	Every 4 vertices form one quadrilateral

CONVEX ONLY

OpenGL fills a polygon correctly only if it is convex. Concave shapes must be split into convex pieces such as triangles.

PART B · CHAPTER 4

Attributes of Graphics Primitives

Attributes are parameters that control how a primitive is displayed, such as its colour, size, thickness and style.

Colour and Gray Scale

Colour

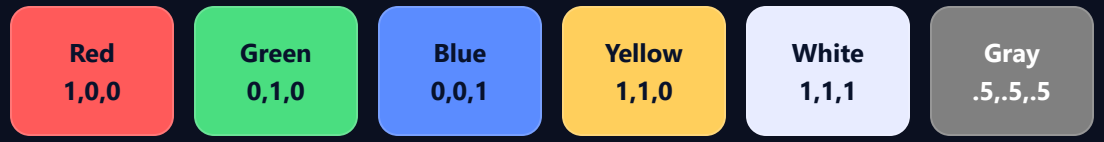
- **RGB model**: a colour is a mix of red, green and blue.
- **Direct or true colour**: each pixel stores its full RGB value.
- **Colour lookup table**: a pixel stores an index into a palette.

```
glColor3f(1.0, 0.0, 0.0); // red
```

Gray scale

- Shades of gray on monochrome systems.
- Equal R, G and B gives gray; the value sets brightness.
- Levels run from black (0) to white (max).

```
glColor3f(0.5, 0.5, 0.5); // mid gray
```



Point Attributes

Two things control how a point looks

- **Colour**, set by the current colour state.
- **Size**: a point can be drawn larger than a single pixel, as a small square.

OPENGL POINT SIZE

`glPointSize(size)` sets the point diameter in pixels (default 1.0). Call it before `glBegin(GL_POINTS)`.

glPointSize grows the point.

Line Attributes

Colour, width and style define how lines look

Line width

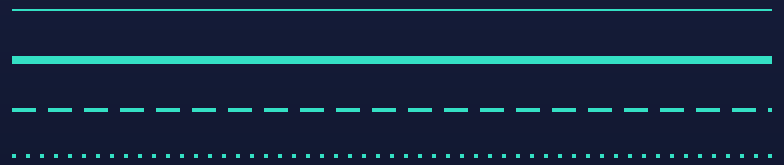
Thickness of the line in pixels.

Line style

Solid, dashed, dotted, dash dot patterns.

Line colour

Set by the current colour state.



Thin, thick, dashed and dotted lines.

Dashed lines use a **pixel mask** (bit pattern) along the line: where the mask bit is 1 the pixel is drawn, where it is 0 it is skipped, creating the dashes.

Curve Attributes

Curves share the same attribute parameters as lines

- **Colour**: the current colour applies to the whole curve.
- **Width or thickness**: set like line width.
- **Style**: dashed or dotted patterns applied along the curve.

> **BECAUSE CURVES ARE POLYLINES**
>
> In OpenGL a curve is a sequence of short segments, so the same `glLineWidth` and stipple attributes apply to it automatically.

OpenGL Point Attribute Functions

```
glPointSize(3.0);          // 3 pixel wide points
glColor3f(0.0,1.0,0.0);    // green

glBegin(GL_POINTS);
    glVertex2i(50,50);
    glVertex2i(100,80);
glEnd();
```

- `glPointSize(GLfloat size)` sets the point size in pixels.
- `glColor3f`, `glColor3i`, `glColor4f` set the current colour.
- Attributes are **state**: they stay in effect until you change them.

OpenGL Line Attribute Functions

```
glLineWidth(2.0);                 // 2 pixel wide lines

glEnable(GL_LINE_STIPPLE);         // turn on dashed lines
glLineStipple(1, 0x00FF);        // factor, 16 bit pattern

glBegin(GL_LINES);
    glVertex2i(20,20);
    glVertex2i(180,120);
glEnd();

glDisable(GL_LINE_STIPPLE);
```

- `glLineWidth(GLfloat w)` sets the line thickness.
- `glLineStipple(factor, pattern)` defines a dash pattern; the 16 bit mask is stretched by the factor.
- Enable with `glEnable(GL_LINE_STIPPLE)` and disable when done.

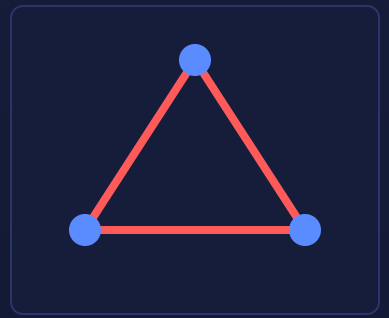
Putting it Together

A thick red triangle outline with big blue points at the corners

```
void display(void) {
    glClear(GL_COLOR_BUFFER_BIT);

    // red triangle outline, 3px wide
    glColor3f(1,0,0);
    glLineWidth(3.0);
    glBegin(GL_LINE_LOOP);
        glVertex2i(50,50);
        glVertex2i(150,50);
        glVertex2i(100,140);
    glEnd();

    // big blue corner points
    glColor3f(0,0,1);
    glPointSize(8.0);
    glBegin(GL_POINTS);
        glVertex2i(50,50);
        glVertex2i(150,50);
        glVertex2i(100,140);
    glEnd();
    glFlush();
}
```



The result of the code on the left.

Module 2 Summary

Output Primitives

- Coordinate frames run from world space to device space.
- `gluOrtho2D` defines the 2D world window.
- Points use `GL_POINTS`; lines use LINES, STRIP or LOOP.
- Curves are approximated by line segments; GLU handles smooth surfaces.

Attributes

- Colour (RGB) and gray scale via `glColor*`.
- Point size via `glPointSize`.
- Line width via `glLineWidth`; style via `glLineStipple`.
- Attributes are persistent OpenGL state.

NEXT UP

Module 3, line drawing algorithms and 2D and 3D transformations, Chapters 5, 6 and 8.

Sources & References

Textbook chapters and official documentation for this module

- Hearn, Baker & Carithers, **Computer Graphics with OpenGL**, 4th ed., Pearson, 2014, **Chapter 3** (Graphics Output Primitives) and **Chapter 4** (Attributes of Graphics Primitives).
- OpenGL point, line and colour commands: OpenGL / GLU reference pages, Khronos OpenGL Registry.
- Reference: Edward Angel, **Interactive Computer Graphics**, 5th ed., Pearson, 2008.

LECTURER

Mr. Bharath · B24CS53 Computer Graphics.